Task 2

Exercise 4: Order Items Join Optimization

-- A) Optimization method

- without index we will make full table scan — scanning every row to find matching order items. --An index on order_id lets the database engine jump directly to the relevant rows using a tree structure, -- reducing lookup time from O(n) to O(log n).

B) Applying the optimization — creating the index

- This tells the database to maintain a index structure on order_id, -- which both join lookups and WHERE order_id = ? filters can exploit instantly.

```sql
-- B) Applying the optimization - creating the index
-- This tells the database to maintain a index structure on order_id,
-- which both join lookups and WHERE order_id = ? filters can exploit instantly.
CREATE INDEX idx_order_items
ON sales.order_items (order_id)

```

without index

```sql
-- C) Query to fetch order item details for a given order_id
-- without index
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
SELECT oi.order_id, oi.product_id, oi.quantity, oi.list_price, oi.discount
FROM sales.orders o
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_id = 5;

-- مع index (عادي)
SELECT oi.order_id, oi.product_id, oi.quantity, oi.list_price, oi.discount
```

% 5 0 Ln: 18 Ch: 17 SPC CRI

Results | Messages

```
 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(3 rows affected)
Table 'order_items'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logica
Table 'orders'. Scan count 0, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical rea

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-27T23:27:34.6703635+02:00
```

with index

```sql
-- with index
SELECT oi.order_id, oi.product_id, oi.quantity, oi.list_price, oi.discount
FROM sales.orders o
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_id = 5;

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
```

9 % 5 0 Ln: 27 Ch: 1 SPC CRLF

Results | Messages

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 11 ms.

(3 rows affected)
Table 'order_items'. Scan count 1, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logica
Table 'orders'. Scan count 0, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical rea

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Completion time: 2026-03-27T23:28:41.7082365+02:00
```

Exercise 5: Store Revenue View

A) Implement a solution that allows quick revenue calculations (create view)

- the best approach here is to create a VIEW — a saved query stored in the database that behaves like a virtual table.
- It does not store data itself; it runs the underlying query each time it is called.

SQLQuery1.sq...Shield (73))*

```sql
-- Exercise 5: Store Revenue View
-- A) Implement a solution that allows quick revenue calculations (create view)
-- the best approach here is to create a VIEW – a saved query stored in the database that behaves lik
-- It does not store data itself; it runs the underlying query each time it is called.
CREATE VIEW
sales.store_revenue1 AS
SELECT
    s.store_name,
    SUM(oi.list_price * oi.quantity) AS total_revenue
FROM sales.stores s
JOIN sales.orders o
    ON s.store_id = o.store_id
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
GROUP BY s.store_name;
```

119 % 1 0 Ln: 44 Ch: 1 SPC CR

Messages

```
Commands completed successfully.

Completion time: 2026-03-27T23:39:46.1109949+02:00
```

B) Query — Fetch Total Revenue per Store

- Once the VIEW is created, you query it like a regular table — one clean line instead of rewriting the joins every time
- The ORDER BY DESC sorts results from highest to lowest revenue

SQLQuery1.sq...Shield (73))*

```sql
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
GROUP BY s.store_name;


-- B) Query - Fetch Total Revenue per Store
-- Once the VIEW is created, you query it like a regular table - one clean line instead of rewriting the joins e
-- The ORDER BY DESC sorts results from highest to lowest revenue
SELECT
    store_name,
    total_revenue
FROM sales.store_revenue
ORDER BY total_revenue DESC
```

109 % 5 0 Ln: 60 Ch: 1 SPC CRLF

Results | Messages

	store_name	total_revenue
1	Baldwin Bikes	5826242.21
2	Santa Cruz Bikes	1790145.91
3	Rowlett Bikes	962600.76